

Project Title: Apollo 24/7 – Lab Test Module Automation

Module: Popular Health Checkup + Complete Checkout

1. Executive Summary

This document serves as the formal test execution report for the Apollo 24/7 Lab Test Checkout flow automation project. The objective of this project is to implement robust, end-to-end automated testing for the Popular Health Checkup module to ensure functional correctness, data integrity, and a seamless user experience.

2. Project Description

The automation project provides a comprehensive end-to-end testing solution for the Apollo 24/7 web application, explicitly targeting the Lab Tests module. It encompasses the complete checkout flow for Popular Health Checkups, dynamically verifying critical application features such as package card visibility, dynamic pricing display (including the '₹' symbol), and Call-to-Action (CTA) functionality.

3. Test Strategy

The test framework is architected to ensure maximum scalability, maintainability, and reliability:

- **Design Pattern:** Implemented the Page Object Model (POM) to abstract UI element interactions from the core business logic, ensuring code reusability.
- **Data-Driven Testing:** Integrated CSV-based parameterization to execute tests iteratively across 8 distinct health packages without hardcoding parameters.
- **Robust Locators:** Utilized multi-strategy locators with prioritized XPath fallbacks to proactively manage A/B UI variations on the Apollo 24/7 platform.
- **Resilient Interstitial Handling:** Implemented a strategic three-tier fallback mechanism (Escape Key -> Close Button -> Backdrop Click) to dismiss unpredicted popups and modals.
- **Scope of Testing:** Executed 13 comprehensive test cases categorized into 12 Positive tests (verifying expected functional behavior) and 1 Negative test (verifying constraints).

4. Test Environment

The automated testing suite was executed under the following standardized environment configurations:

- **Programming Language:** Python 3.x
- **Core Libraries:** Selenium WebDriver, Pytest, BDD Behave
- **Reporting Framework:** Allure Framework
- **Target Application URL:** <https://www.apollo247.com/>
- **Configured Browser:** Microsoft Edge (configurable to Google Chrome via properties)
- **Synchronization Strategy:** 10-second implicit wait mechanism, complemented by dynamic explicit WebDriverWait functions for specific elements.

- **Dependency Management:** Maintained consistently through a virtual environment utilizing requirements.txt.

5. Test Execution

The execution phase was standardized to ensure repeatability:

- **Execution Engine:** Tests were orchestrated using the pytest runner. Specific command-line execution (pytest tests/test_popular_health_checkup.py -v) was utilized for granular testing.
- **Continuous Logging:** Daily rotating logs (e.g., automation_YYYYMMDD.log) recorded verbose, step-by-step system events and exceptions throughout the runtime.
- **Failure Handling:** Automated screenshot capture was triggered on test assertion failures to preserve the visual state of the application for debugging.

6. Test Output

Systematic outputs were generated to facilitate test analysis and traceability:

- **Real-time Console Output:** Provided immediate feedback on test progression, passing rates, and failure points.
- **Application Logs:** Structured logs saved in the logs/ directory, detailing driver initialization, configuration loading, and teardown states.
- **Visual Evidences:** Screenshots of application state upon failure stored securely in the reports/screenshots/ directory.

7. Test Reports

Comprehensive post-execution reports were generated to summarize testing quality metrics:

- **HTML Summary Report:** A consolidated, standalone Pytest HTML report generated at reports/report.html detailing individual test statuses and durations.
- **Allure Analytical Dashboard:** An interactive metrics dashboard capturing Epic/Feature/Story classifications, visual timelines, and attached failure screenshots. This report is accessible via the command allure serve reports/allure-results.